Chapitre 3:
Algorithmes de tri

Chapitre3: Algorithmes de tri

Introduction

Définition:

Étant donné un tableau d'entiers T (n: sa taille), l'algorithme de tri permet d'organiser les éléments du tableau selon un ordre déterminé (croissant par exemple).

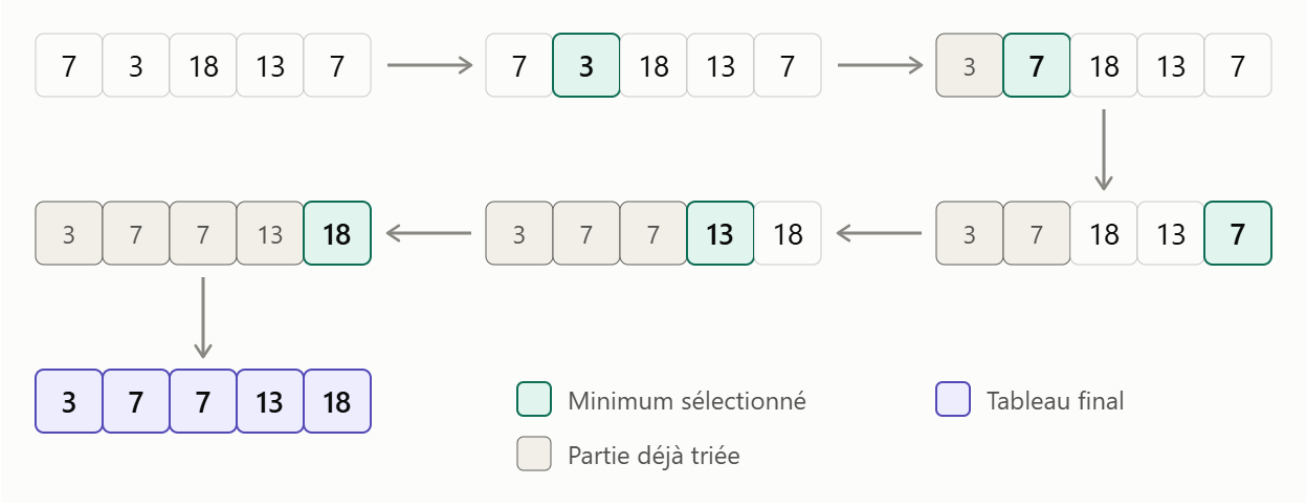
Plusieurs algorithmes de tri existent:

- tri par sélection,
- par insertion,
- par bulle,
- par fusion,
- rapide,
- par TAS.

Tri par sélection

Principe:

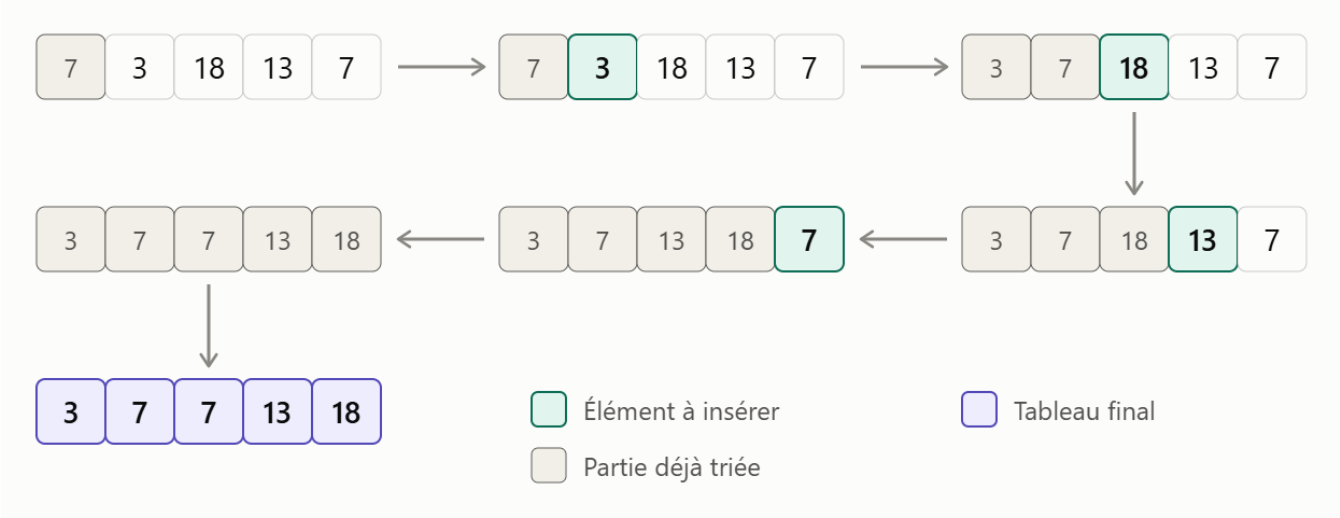
Rechercher la plus petite valeur et la placer au début du tableau, puis la plus petite valeur dans les valeurs restantes et la placer à la deuxième position et ainsi de suite...



Tri par insertion

Principe:

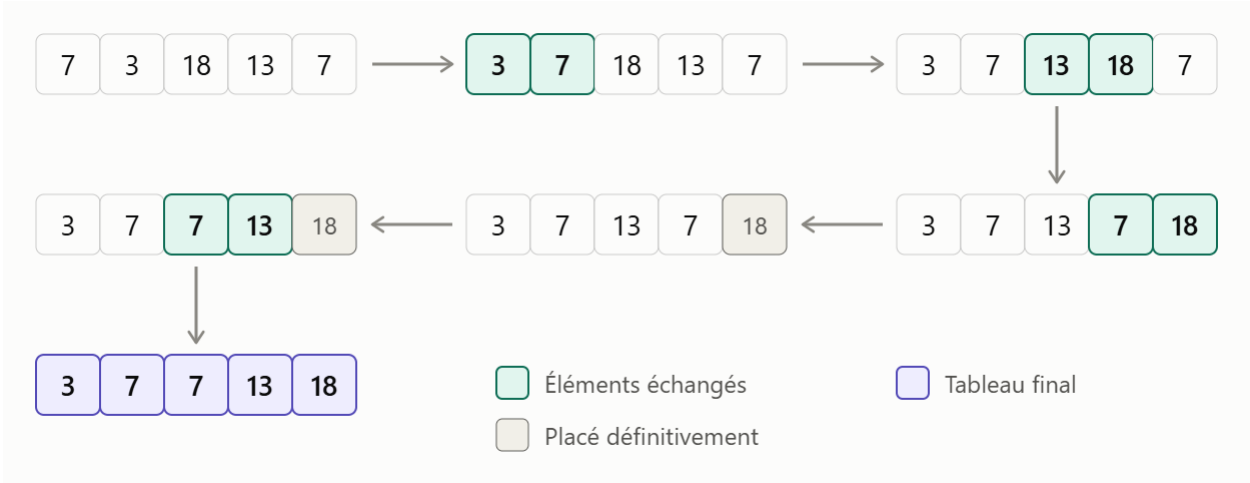
ordonner les deux premiers éléments et d'insérer le 3ème élément de manière à ce que les 3 premiers éléments soient triés, ensuite d' insérer le 4ème élément à sa place et ainsi de suite. A la fin de la ième itération, les i premiers éléments de T sont triés et rangés au début du tableau T'



Tri à bulles

Principe:

L'algorithme de tri à bulles consiste à faire remonter progressivement les plus grands éléments d'un tableau. Son principe est d'inverser deux éléments successifs s'ils ne sont pas classés dans le bon ordre et de recommencer jusqu'à ce qu'on ne peut plus permuter.

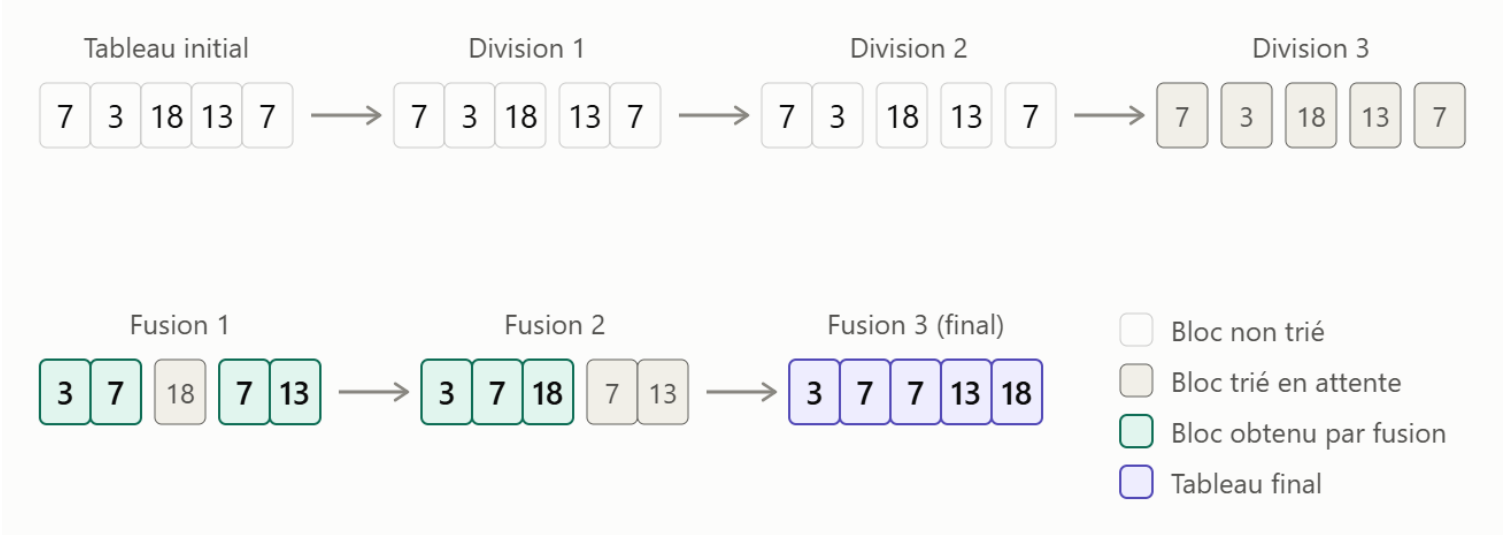


Tri par fusion

Principe:

Le tri fusion utilise la stratégie dite “diviser pour régner”.

- Etape 1 : On divise le tableau en deux sous-tableaux.
- Etape 2 : On trie (par récursivité) chaque sous-tableau.
- Etape 3 : On fusionne les deux sous-tableaux, pour obtenir un tableau complet trié.



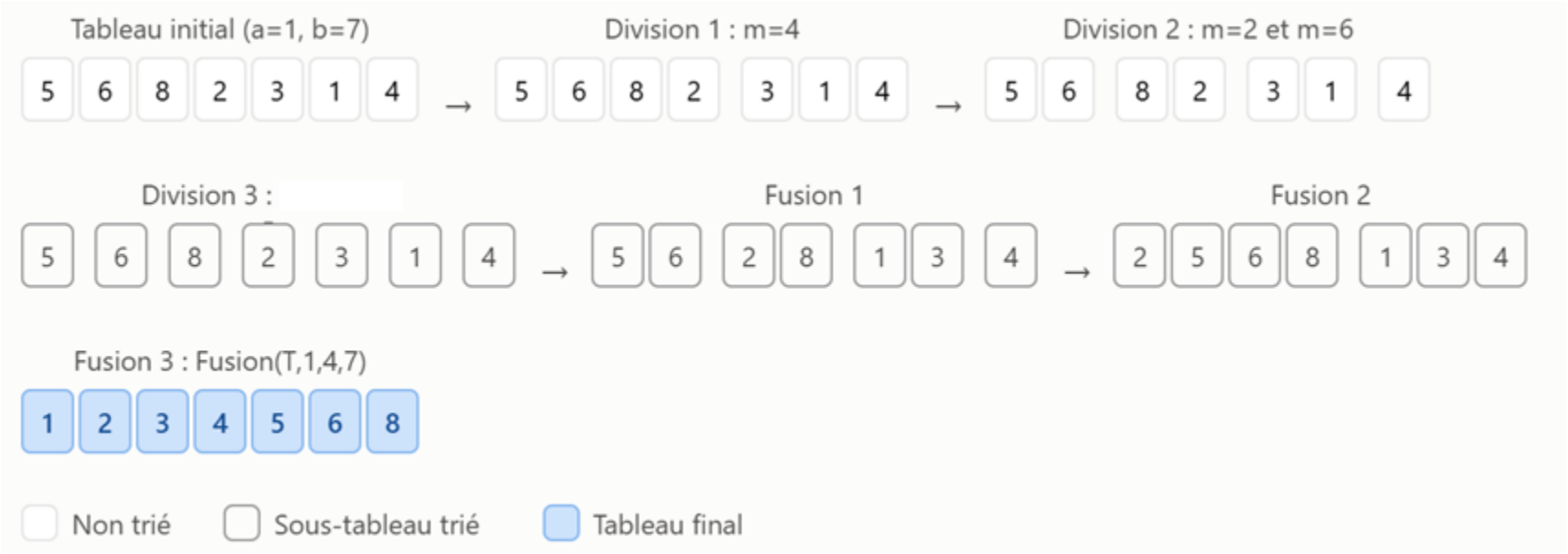
Tri par fusion

```
Algorithme: TriFusion(T,a,b)
si (a < b ) alors
        m = ⌊(a+b)/2⌋;
        TriFusion(T,a,m);
        TriFusion(T,m+1,b);
        Fusion(T,a,m,b);
fin
```

```
Algorithme: Fusion(T,a,m,b)
n1 = m-a+1 ; n2 = b-m;
créer les tableaux G[1...n1+1] et D[1...n2+1] ;
pour i allant de 1 à n1 faire
        G[i] = T[a+i-1] ;
fin
pour j allant de 1 à n2 faire
        D[j] = T[m+j] ;
fin
G[n1+1] = ∞, D[n2+1] = ∞ ;
i = 1 ; j = 1 ;
pour k allant de a à b faire
        si G[i] ≤ D[j] alors
                T[k] = G[i] ; i = i+1 ;

        sinon
                T[k] = D[j] ; j = j+1 ;
        fin
fin
```

Tri par fusion



Tri par fusion

Soit C(n) le nombre total, au pire des cas, des comparaisons indiquées en rouge. On a :

$$C(n) = C(\lfloor n/2 \rfloor) + C(\lceil n/2 \rceil) + n$$

Théorème:

Si n est une puissance de 2 alors: $C(n) = 2C(n/2)+n = n \log_2(n)$

Tri rapide

Principe:

Soit T un tableau à trier. On se donne un élément quelconque du tableau (en général le premier à gauche) nommé pivot. On effectue une partition du tableau consistant à réorganiser les éléments de T de la façon suivante :

$$T[1...m-1], T[m], T[m+1...n]$$

- T[m] contient le pivot choisi.
- T[1...m − 1] contient les éléments de T inferieurs à T[m].
- T[m + 1...n] ceux supérieurs.
- Sur chaque sous-tableau, on réitère récursivement ce qui a été fait sur T.

Tri rapide

```
Algorithme: TriRapide(T,a,b)
si (a < b ) alors
        Partitionnement(T,a,b,m);
        TriRapide(T,a,m-1);
        TriRapide(T,m+1,b);

fin
```

Tri rapide

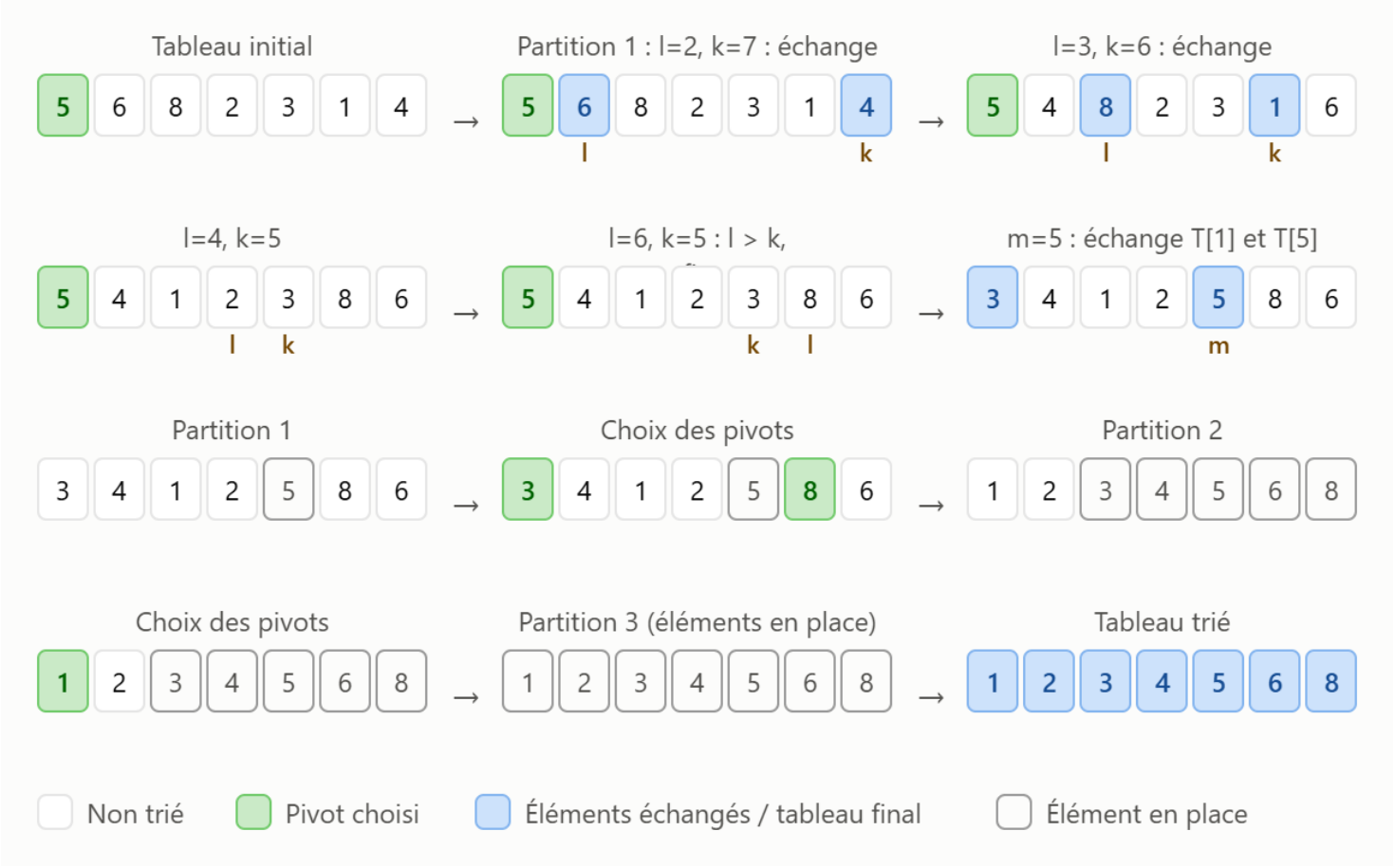
Principe de partitionnement:

- On considère comme pivot le premier élément du tableau en cours de traitement, **T[a]**.
- On utilise deux compteurs **l** et **k** initialisés aux deux extrémités (gauche pour **l**, et droite pour **k**) du tableau.
- Tant que **T[l]** ≤ **T[a]**, **l** est incrémenté.
- Inversement tant que **T[k]** > **T[a]**, **k** est décrémenté.
- On échange alors **T[l]** et **T[k]**
- On continue de façon analogue jusqu'à ce que les indices **l** et **k** se croisent.

Tri rapide

```
Algorithme: Partitionnement(T,a,b,m)
l = a + 1 ;
k = b ;
tant que l ≤ k faire
  tant que T[l] ≤ T[a] faire
     l = l + 1 ;
  fin
  tant que T[k] > T[a] faire
     k = k - 1 ;
  fin
  si (l < k) alors
     echanger(T[l], T[k]) ;
     l = l + 1 ;
     k = k - 1 ;
  fin
fin
m = k ;
echanger(T[a], T[k]) ;
fin
```

Tri rapide


Tableau initial
5 6 8 2 3 1 4
Partition 1 : l=2, k=7 : échange
5 6 8 2 3 1 4
l
k
l=3, k=6 : échange
5 4 8 2 3 1 6
l
k
l=4, k=5
5 4 1 2 3 8 6
l k
l=6, k=5 : l > k,
5 4 1 2 3 8 6
k l
m=5 : échange T[1] et T[5]
3 4 1 2 5 8 6
m
Partition 1
3 4 1 2 5 8 6
Choix des pivots
3 4 1 2 5 8 6
Partition 2
1 2 3 4 5 6 8
Choix des pivots
1 2 3 4 5 6 8
Partition 3 (éléments en place)
1 2 3 4 5 6 8
Tableau trié
1 2 3 4 5 6 8
Non trié
Pivot choisi
Éléments échangés / tableau final
Élément en place

Tri rapide

```
Algorithme: Partitionnement(T,a,b,m)
l = a + 1 ;
k = b ;
tant que l ≤ k faire
  tant que T[l] ≤ T[a] faire
     l = l + 1 ;
  fin
  tant que T[k] > T[a] faire
     k = k - 1 ;
  fin
  si (l < k) alors
     echanger(T[l], T[k]) ;
     l = l + 1 ;
     k = k - 1 ;
  fin
fin
m = k ;
echanger(T[a], T[k]) ;
fin
```

Nombre de comparaisons.

Il y a $b - a + 2$ comparaisons.